Softwareentwicklung

Skriptum zur Vorlesung - 01.09.2023

Dipl.-Ing. Paul Panhofer BSc.[1*]

1 ZID, TU Wien, Taubstummengasse 11, 1040, Wien, Austria

Abstract:

MSC: p.panhofer@htlkrems.at

Keywords:

Contents

1. Programmierung: Strukturierung 4
1.1. Unterprogramme 4
1.1.1. Unterprogramme 4
1.2. Objektorientierung 5
1.2.1. Objektorientierung 5
1.3. Schichtenmodell 5
1.3.1. Prinzipien des Schichtenmodells 5
1.3.2. Fallbeispiel: Schichtenmodell 6
1.4. Komponenten 8
1.4.1. Fallbeispiel: Restaurantverwaltung 8
1.5. Service 9
1.5.1. Zusammenfassung 9

Index 10

*E-mail: paul.panhofer@tuwien.ac.at

Softwareentwicklung - **Theorieskriptum**

Grundlagen der objektorientierten Programmierung

December 14, 2019

1. Programmierung: Strukturierung

01

Sturkturierung von Programmen

01.	Unterprogramme	4
02.	Objektorientierung	5
03.	Schichtenmodell	5
04.	Komponenten	8
05.	Service	9

1.1. Unterprogramme

Historisch gesehen hat alles mit einem bunten Gemisch aus **Anweisungen** und **Daten** innerhalb eines Betriebssystemprozesses[1] begonnen. Der **Prozess** spannte die Laufzeitumgebung für den Code auf. Programme waren zu dieser Zeit kurz und einfach.

> Die kleinste Einheit eines Programms war die **Anweisung**.

1.1.1 Unterprogramme

Die zunehmende **Codekomplexität** von Softwareanwendungen verlangte nach neuen Wegen Code zu strukturieren.

Erklärung: Unterprogramme

- **Unterprogramme**[2] entstanden als Programme umfangreicher wurden.
- Sie waren ein erster Schritt zur **Kapselung** von Code.
- Die Zahl der Anweisungen pro Anwendung konnten ansteigen, ohne dass die **Wartbarkeit**[3] der Anwendung gesunken wäre.
- Als nächstes wurden **Container** für Daten[4] entwickelt.

Codebeispiel: Unterprogramme

```
struct Point3D {
   double x,y,z;
};
main(){
   settextstyle(BOLD_FONT,HORIZ_DIR,2);
   x = getmaxx()/2;
   y = getmaxy()/2;

   return 0;
}
```

[1] *Unter einem Betriebssystemprozess verstehen wir ein sich in Ausführung befindendes Programm*
[2] *Funktionen, Prozeduren*
[3] *Codeerwartbarkeit, Codelesbarkeit, Anpassbarkeit*
[4] *Die Sprache C spiegelt diesen Entwicklungsstand wider: sie bietet Unterprogramme (Prozeduren und Funktionen) sowie Strukturen zur Strukturierung*

1.2. Objektorientierung

Der nächste Schritt in der Evolution der Anwendungsprogrammierung war das objektorientierte Programmierparadigma.

1.2.1 Objektorientierung

Objektorientierung faßt Strukturen und Unterprogramme zu **Klassen**[5] zusammen. Dadurch wurde Software nochmal etwas grobgranularer, so dass sich mehrere Anweisungen innerhalb eines Prozesses verwalten ließen.

> Die kleinste Einheit eines objektorientierten Programms ist die **Klasse**.

▸ **Erklärung: Klasse** ▾

- Eine Klasse stellt **Funktionalität**[6] zur Verfügung, die den **Zustand**[7] von Instanzen der Klasse verändert und verarbeitet.
- Variablen und Methoden stehen im kontinuierlichen Zusammenspiel.

▸ **Codebeispiel: Klassen** ▾

```
// ----------------------------------------
//   Project.cs
// ----------------------------------------
[Table("PROJECTS")]
public class Project : AProject {

    [Key, DatabaseGenerated]
    [Column("PROJECT_ID")]
    public int Id { get; set; }

    [Required, StringLength(50)]
    [Column("TITLE")]
    public string Title { get; set;}

    public Project() : base () {

    }
}
```

❑

[5] *Die hauptsächliche **Strukturierung** von Software befindet sich heute auf dem Niveau der **1990er**, als die Objektorientierung mit* C++, Delphi *und dann* Java *ihren Siegeszug angetreten hat.*
[6] *Methoden*
[7] *Variblen*

1.3. Schichtenmodell

Das Schichtenmodell ist ein häufig angewandtes Strukturierungsprinzip für die **Architektur** von Softwaresystemen. Dabei werden einzelne logisch zusammengehörende **Aspekte** des Softwaresystems konzeptionell einer **Schicht** zugeordnet.

1.3.1 Prinzipien des Schichtenmodells

▸ **Prinzip: Schichtenmodell** ▾

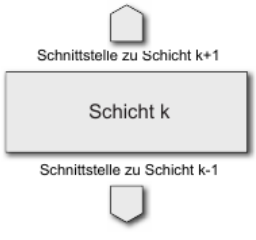
- **Teile und Herrsche:** Ein komplexes Problem wird in **unabhängige Teilprobleme** zerlegt, das jedes für sich, einfacher handhabbar ist, als das Gesamtproblem.

  Oft ist es erst durch die Fromulierung von Teilproblemen möglich, ein komplexe Probleme zu lösen.

- **Unabhängigkeit:** Die einzelnen Schichten der Anwendung kommunizieren miteinander, indem die **Schnittstellenspezifikation**[8] des direkten Vorgängers bzw. Nachfolgers genutzt wird.

  Durch die **Entkoppelung** der Spezifikation der Schicht von ihrer **Implementierung** werden Abhängigkeiten zwischen den Schichten vermieden.

- **Abschirmung:** Eine Schicht kommuniziert ausschließlich mit seinen benachbarten Schichten. Damit wird eine **Kapselung** der einzelnen Schichten erreicht, wodurch die zu bewältigende **Komplexität** sinkt.



- **Standadisierung:** Die Gliederung des Gesamtproblems in einzelne Schichten erleichtert die Entwicklung von **Standards**[9] für die einzelnen Schichten.

❑

[8] *Schnittstelle, Interface*
[9] *HTTP, FTP, usw.*

1.3.2 Fallbeispiel: Schichtenmodell

Schnittstellenspezifikation: Modelschicht

```
//-----------------------------------------
//  AosDbcontext.cs
//-----------------------------------------
public class AosDbContext : DbContext {

    public DbSet<Trait> Traits { get; set; }
    public DbSet<TraitItem> TItems {get;set;}

    public AosDbContext(
      DbContextOptions<AosDbContext> options)
          : base(options
    ) { }

    protected override void
        OnModelCreating(ModelBuilder builder)
        {
        builder.Entity<Attack>()
            .HasIndex(a => a.Identifier)
            .IsUnique();

        builder.Entity<Attack>()
            .HasOne(a => a.Creature)
            .WithMany()
            .HasForeignKey(a => a.CreatureId);

        builder.Entity<Attack>()
            .Property(a => a.AttackType)
            .HasConversion<string>();

        builder.Entity<Trait>()
            .HasIndex(t => t.Identifier)
            .IsUnique();

        builder.Entity<TraitItem>()
            .HasKey(ti => new {ti.CreatureId,
                ti.TraitId});

        builder.Entity<TraitItem>()
            .HasOne(ti => ti.Creature)
            .WithMany()
            .HasForeignKey(ti =>
                ti.CreatureId);

        builder.Entity<TraitItem>()
            .HasOne(ti => ti.Trait)
            .WithMany()
            .HasForeignKey(ti => ti.TraitId);
    }
}
```

Schnittstellenspezifikation: Domainschicht

```
//-----------------------------------------
//  IRepository.cs, AReposiotry.cs
//-----------------------------------------
public interface IRepository<TEntity> where
    TEntity : class {

    TEntity Create(TEntity t);

    List<TEntity> CreateRange(List<TEntity>
        list);

    void Update(TEntity t);

    void UpdateRange(List<TEntity> list);

    TEntity? Read(int id);

    List<TEntity>
        Read(Expression<Func<TEntity, bool>>
        filter);
}

public abstract class ARepository<TEntity> :
    IRepository<TEntity> where TEntity :
    class {

    protected readonly AosDbContext Context;

    protected readonly DbSet<TEntity> Table;

    protected ARepository(AosDbContext
        context) {
        Context = context;
        Table = context.Set<TEntity>();
    }

    public TEntity Create(TEntity t) {
        Table.Add(t);
        Context.SaveChanges();

        return t;
    }

    public List<TEntity>
        CreateRange(List<TEntity> list) {
        Table.AddRange(list);
        Context.SaveChanges();

        return list;
    }

    public void Update(TEntity t) {
```

```
        Context.ChangeTracker.Clear();

        Table.Update(t);
        Context.SaveChanges();
    }

    public void UpdateRange(List<TEntity> list) {
        Table.UpdateRange(list);
        Context.SaveChanges();
    }

    public TEntity? Read(int id) => Table.Find(id);

    public List<TEntity> Read(Expression<Func<TEntity, bool>> filter) =>
        Table.Where(filter).ToList();

    public List<TEntity> Read(int start, int count) =>
        Table.Skip(start)
            .Take(count)
            .ToList();

    public List<TEntity> ReadAll() => Table.ToList();

    public void Delete(TEntity t) {
        Table.Remove(t);
        Context.SaveChanges();
    }
}
```

Schnittstellenspezifikation: Serviceschicht

```
//-----------------------------------------
//  AController.cs
//-----------------------------------------
public class AController<TEntity> : ControllerBase where TEntity : class {

    private IRepository<TEntity> _repository;

    private ILogger<AController<TEntity>> _logger;

    public AController(
      IRepository<TEntity> repository,
      ILogger<AController<TEntity>> logger
    ) {
        _repository = repository;
        _logger = logger;
    }

    [HttpPost]
    public async Task<ActionResult<TEntity>> Create(TEntity t) {
        await _repository.CreateAsync(t);
        _logger.LogInformation($"Created entity with id: {t}");

        return t;
    }

    [HttpGet("{id:int}")]
    public async Task<ActionResult<TEntity>> Read(int id) {
        var data = await _repository.ReadAsync(id);

        if (data is null) return NotFound();
        _logger.LogInformation($"reading entity with id {id}");

        return Ok(data);
    }

    [HttpGet]
    public async Task<ActionResult<List<TEntity>>> ReadAll(int start, int count) => Ok(await _repository.ReadAllAsync(start, count));

    [HttpPut("{id:int}")]
    public async Task<ActionResult> Update(int id, TEntity entity) {
        var data = await _repository.ReadAsync(id);

        if (data is null) return NotFound();

        await _repository.UpdateAsync(entity);
        _logger.LogInformation($"updated entity: {entity}");
        return NoContent();
    }
}
```

❑

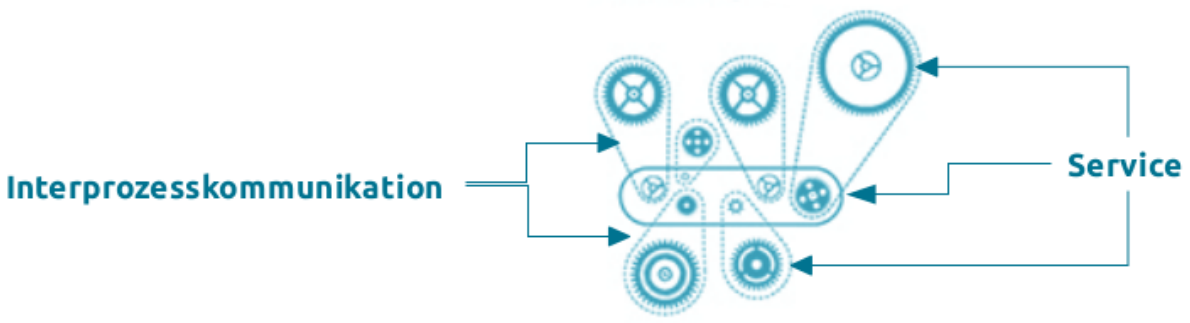


Abbildung 1. **SOA - Zusammenspiel von Services**

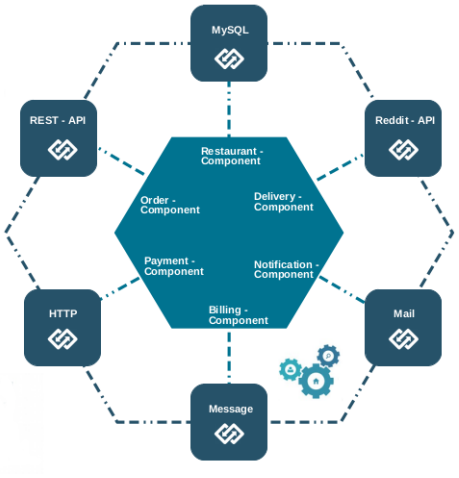
1.4. Komponenten

Bei der Entwicklung von Softwareanwendungen besteht die erste Aufgabe der Softwareentwickler darin, die voneinander unabhängigen Teile der **Anforderungsbeschreibung** voneinander zu isolieren. Wir nennen diese Teile **Komponenten** bzw. Module in der Softwareentwicklung.

> **Komponenten** werden in **Schichten** unterteilt. Jede Schicht wiederum besteht aus **Klassen**.

Erklärung: Komponente

- Komponenten definieren sich als von einander **unabhängige Teile** der Anforderungsbeschreibung eines Systems.
- Für die Kommunikation stellen Komponenten **Schnittstellen** zur Verfügung.



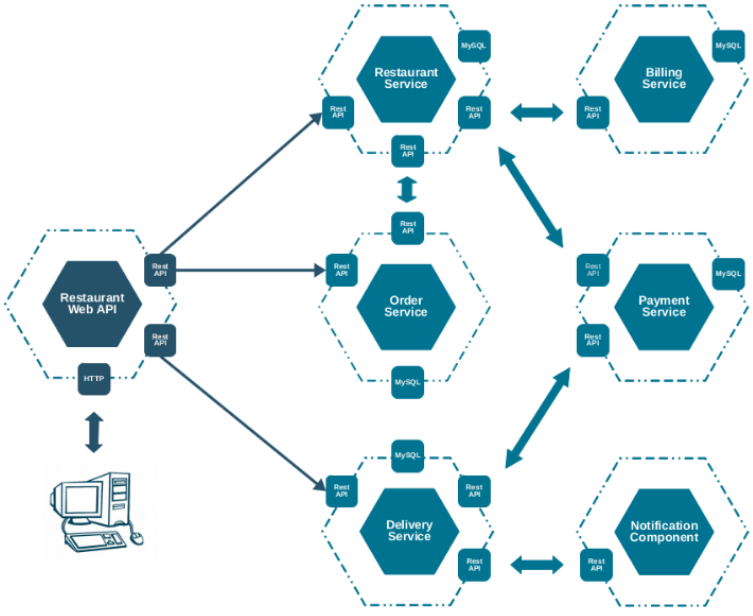
1.4.1 Fallbeispiel: Restaurantverwaltung

Fallbeispiel: Restaurantverwaltungssoftware

- Es soll eine Restaurantverwaltungssoftware entwickelt werden.
- Als erstes **isolieren** wir die einzelnen **Komponenten** voneinander.
- **Komponenten** der Restaurantverwaltungssoftware:
  - **Restaurantkomponente:** Lokalbesitzer benutzen die Funktionalität der Restaurantkomponente um die Speisekarte für ihre Lokale zu verarbeiten.
  - **Orderkomponente:** Benutzer platzieren Bestellungen über eine Homepage bzw. Smartphoneanwendung Bestellungen in bestimmten Lokalen. Die Orderkomponente stellt dazu die Funktionalität zur Verfügung.
  - **Deliverykomponente:** Die Anwendung erlaubt es einer Reihe von Kurierdiensten Bestellungen auszuliefern. Die Deliverykomponente hilft bei der Verwaltung der Bestellungen.
  - **Notificationkomponente:** Die Anwendung verschickt Benachrichtigungen an die Lokale und Kunden. Die Funktionalität dafür wird von der Notificationkomponente umgesetzt.
  - **Billingkomponente:** Die Billingkomponente wird eingesetzt um die Abrechnung der Bestellung der Kunden druchführen zu können.
- Die einzelnen Komponenten können nun **unabhängig** voneinander entwickelt werden.

❑

Microservice – Architektur



1.5. Service

Service

Ein **Service** ist eine **Softwarekomponente** die in einem eigenen Betriebssystemprozess ausgeführt wird.

> In einer **SOA Anwendung** bzw. in einer **Microsystemanwendung** ist das **Service** die kleinste Strukturierungseinheit der Anwendung.

Analyse: Service

- Komplexe Softwareanwendungen verteilen ihre **Geschäftslogik** auf mehrere Service.
- Ein **Service** definiert unabhängig von seiner Implementierung eine **Schnittstelle**. Der Zugriff auf das Service erfolgt exklusiv über diese Schnittstelle.
- Die **Servicekommunikation** erfolgt über ein Technologie unabhängige Protokolle.
- Die Service einer Softwareanwendung können in unterschiedlichen Technologien implementiert werden.

❑

1.5.1 Zusammenfassung

Qualität und **Kosten** der Erstellung von Softwareanwendungen hängen entscheidend von der **Codekomplexität** ab. **Fehleranzahl** und **Robustheit** eines Codes stehen in engem Zusammenhang zur Softwarekomplexität.

> Zur **Senkung** der **Codekomplexität** wurden unterschiedliche Methoden zur **Strukturierung** von Code entwickelt.

Analyse: Codestrukturierung

- **Softwareanwendungen** bestehen aus **Services**. Ein Service ist eine **Softwarekomponente** in einem eigenen Betriebssystemprozess.
- **Komponeten** bestehen aus **Schichten**. Schichten bestehen aus **Klassen**.
- **Klassen** werden durch **Methoden** strukturiert.

❑

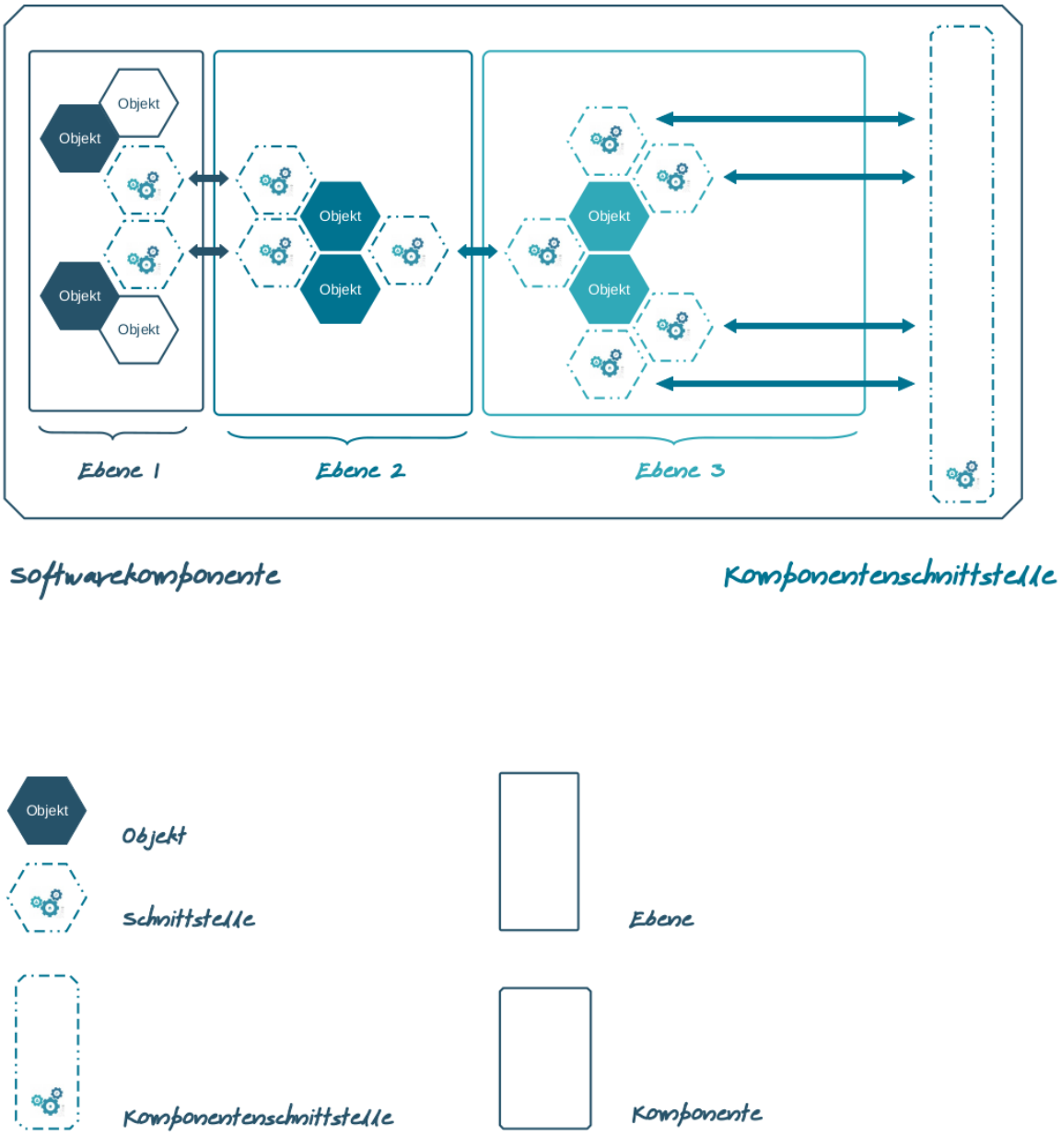


Abbildung 2. **Strukturierung einer Komponente**

.

Index

Architekturmuster, 54

Entwurfsmuster, 54

Idiom, 54

Singleton, 55